

TypeScript Mini Guide

JavaScript with a safety net: the parts you need for Next.js and interviews

TypeScript is JavaScript plus **types**: labels that say what kind of value something is. An editor and the `tsc` checker read those labels and warn you about mistakes **before** you run the code. When the code runs, the types are simply removed; it's plain JavaScript again.

- Part 1** How TypeScript works, and how to practise
- Part 2** The core: basic types, objects, functions
- Part 3** Unions and narrowing (the most useful idea)
- Part 4** Generics, utility types, unknown
- Part 5** TypeScript in React and Next.js
- Part 6** Reading error messages, exercises, interview questions

Part 1: How it works



IN PLAIN ENGLISH

Types exist only **while you code**. They never check anything at runtime. If an API sends you `{"done": "yes"}` when your type says `done: boolean`, TypeScript can't stop that. That's why your API routes still check `typeof done !== "boolean"`.

1 Practise with the runnable file

Node.js can run TypeScript files directly (it strips the types out, no setup). From the `todo-app` folder:

PowerShell

```
node guide/practice/typescript-basics.mts
```

Output

```
1. Buy milk 3 false [ 'home', 'shopping' ] null
2. Cebu
3. { id: 1, text: 'Learn TS', done: false, createdBy: 'me' }
3b. Ana [ 'delete' ]
4. 5 Hi, Ana! Hello, Ben!
5. #7 ABC
5b. 1
6. got 1 todo(s) | failed: offline
7. 10 a
8. Walk dog { done: true } { all: 3, active: 2, done: 1 }
9. hi
10. 1 todo(s) loaded
```

Node **doesn't check** types; it only removes them. To check, run the TypeScript checker on the whole project:

```
PowerShell
```

```
npx tsc --noEmit
```

No output means no errors. `--noEmit` = check only, don't write any files. Practise by breaking things in the file (e.g. `city = 42;`) and running both commands.

Part 2: The core

2 Basic types

```
TypeScript
```

```
const title: string = "Buy milk";
const count: number = 3;           // no int/float split: all numbers are number
const isDone: boolean = false;
const tags: string[] = ["home", "shopping"]; // array of strings
const note: string | null = null;  // a string OR null
```

```
Output
```

```
1. Buy milk 3 false [ 'home', 'shopping' ] null
```

3 Inference: you rarely write the type

```
TypeScript
```

```
let city = "Manila"; // TypeScript infers: string
city = "Cebu";       // fine
city = 42;           // error: Type 'number' is not assignable to type 'string'.
```

TIP

Let TypeScript infer local variables. **Do** write types for function parameters, function return values, and object shapes you share between files. Those are the boundaries where mistakes sneak in.

4 Object types: type and interface

```
TypeScript
```

```
type Todo = {
  id: number;
  text: string;
  done: boolean;
  dueDate?: string; // ? = optional (may be missing)
  readonly createdBy: string; // can't be reassigned
};
const todo: Todo = { id: 1, text: "Learn TS", done: false, createdBy: "me" };

interface User { id: number; name: string }
interface Admin extends User { permissions: string[] } // Admin = User + more
const admin: Admin = { id: 1, name: "Ana", permissions: ["delete"] };
```

Output

```
3. { id: 1, text: 'Learn TS', done: false, createdBy: 'me' }
3b. Ana [ 'delete' ]
```

	type	interface
Describe an object	Yes	Yes
Unions like <code>"a" "b"</code>	Yes	No
Extend another	<code>A & { more }</code>	<code>extends</code>

Both are fine. A simple rule: use `type` by default (our app does), and `interface` when you want `extends`.

5 Functions

TypeScript

```
function add(a: number, b: number): number { // inputs and output typed
  return a + b;
}
const greet = (name: string, greeting = "Hi"): string => `${greeting}, ${name}!`;
// greeting has a default value, so its type (string) is inferred and it's optional

add(2, 3);           // 5
greet("Ana");        // "Hi, Ana!"
greet("Ben", "Hello");
```

Output

```
4. 5 Hi, Ana! Hello, Ben!
```

A function that returns nothing has return type `void`. An `async` function returning a `Todo[]` has return type `Promise<Todo[]>`, like `findAllTodos()` in your model.

Part 3: Unions and narrowing

The idea you'll use every day.

6 Union types: one of several

TypeScript

```
type Id = number | string;

function formatId(id: Id): string {
  if (typeof id === "number") return `#${id}`; // inside this if: id is number
  return id.toUpperCase();                    // after it: id must be string
}
```

Output

```
5. #7 ABC
```

IN PLAIN ENGLISH

Narrowing means TypeScript follows your `if` checks. After `typeof id === "number"` it *knows* `id` is a number inside that branch, so number methods are allowed. Other checks that narrow: `if (x)` (not null/undefined), `x === "done"`, `"text" in obj`, `Array.isArray(x)`.

7 Literal types: only these exact values

TypeScript

```
type Filter = "all" | "active" | "done";

function filterTodos(todos: Todo[], filter: Filter): Todo[] {
  if (filter === "active") return todos.filter((t) => !t.done);
  if (filter === "done") return todos.filter((t) => t.done);
  return todos;
}
filterTodos(list, "active"); // ok
filterTodos(list, "finished"); // error: not one of the three allowed strings
```

Your editor autocompletes the three options. This is perfect for the "filter" practice exercise in the main guide.

8 Discriminated unions: shapes with a tag

TypeScript

```
type Result =
  | { ok: true; data: Todo[] }
  | { ok: false; error: string };

function describe(result: Result): string {
  if (result.ok) return `got ${result.data.length} todo(s)`; // here: has data
  return `failed: ${result.error}`; // here: has error
}
```

Output

```
6. got 1 todo(s) | failed: offline
```

The shared `ok` field tells TypeScript which shape you have. You *can't* read `result.data` without checking `ok` first, so it's impossible to forget error handling.

Part 4: Generics and friends

9 Generics: types as parameters

TypeScript

```
function first<T>(items: T[]): T | undefined {
  return items[0];
}
const n = first([10, 20]); // T = number → n: number | undefined
const s = first(["a", "b"]); // T = string → s: string | undefined
```

Output

7. 10 a

`<T>` is a placeholder filled in when the function is used. You already use generics everywhere:

Code in your app	Meaning
<code>Todo[] (= Array<Todo>)</code>	An array of Todos
<code>Promise<Todo[]></code>	"Later, you'll get an array of Todos"
<code>pool.query<Todo>(...)</code>	"Each row from this query is a Todo"
<code>create<TodoStore>(...)</code>	"This Zustand store has the TodoStore shape"
<code>useState<Todo[]>([])</code>	"This state is an array of Todos" (needed because <code>[]</code> alone says nothing)

10 Utility types: build new types from old ones

TypeScript

```
type NewTodo    = Omit<Todo, "id">;                // everything except id
type TodoUpdate = Partial<Pick<Todo, "text" | "done">>; // text and/or done, optional
type Counts     = Record<Filter, number>;           // { all, active, done }: numbers

const draft: NewTodo = { text: "Walk dog", done: false, createdBy: "me" };
const update: TodoUpdate = { done: true };
const counts: Counts = { all: 3, active: 2, done: 1 };
```

Output

8. Walk dog { done: true } { all: 3, active: 2, done: 1 }

Utility	Makes
<code>Partial<T></code>	Every field optional (great for "update" data)
<code>Required<T></code>	Every field required
<code>Pick<T, "a" "b"></code>	Only those fields
<code>Omit<T, "a"></code>	All fields except those (great for "create" data with no id yet)
<code>Record<K, V></code>	An object with keys K and values V

11 any vs unknown

TypeScript

```
function parse(json: string): unknown {
  return JSON.parse(json);
}

const data = parse('{"text": "hi"}');
// data.text; ← error: you must check what it is first
if (typeof data === "object" && data !== null && "text" in data) {
  console.log(data.text); // allowed after the checks
}
```

Output

9. hi

WATCH OUT

`any` switches TypeScript **off** for that value: anything goes, no help, no errors. `unknown` means "I don't know yet, make me check". Use `unknown` for data from outside (JSON, APIs, user input). Avoid `any`.

12 Type-only imports

TypeScript

```
import type { Todo } from "@models/todo";
```

Brings in **only the type**; it's erased completely when the code runs. That's how your Zustand store (browser code) safely uses `Todo` from the model file, which also imports the database library. That library never ends up in the browser.

Part 5: TypeScript in React and Next.js

13 Typing component props

components/ToDoItem.tsx (type-checked in a test copy of the app)

```
"use client";

import type { Todo } from "@models/todo";

type TodoItemProps = {
  todo: Todo;
  onToggle: (todo: Todo) => void; // a function prop: takes a Todo, returns nothing
  onDelete: (id: number) => void;
};

export function TodoItem({ todo, onToggle, onDelete }: TodoItemProps) {
  function handleChange(e: React.ChangeEvent<HTMLInputElement>) {
    console.log("checked is now", e.target.checked);
    onToggle(todo);
  }

  return (
    <li>
      <input type="checkbox" checked={todo.done} onChange={handleChange} />
      {todo.text}
      <button onClick={() => onDelete(todo.id)}>x</button>
    </li>
  );
}
```

Now `<TodoItem todo={t} />` without `onToggle` and `onDelete` is an error, and so is a typo like `todo.txt` inside.

14 Common React and Next.js types

Type	Use it for
<code>React.ReactNode</code>	Anything renderable: the type of <code>children</code> (see your <code>layout.tsx</code>)
<code>React.ChangeEvent<HTMLInputElement></code>	The <code>e</code> in an input's <code>onChange</code>
<code>React.FormEvent<HTMLFormElement></code>	The <code>e</code> in a form's <code>onSubmit</code>
<code>React.MouseEvent<HTMLButtonElement></code>	The <code>e</code> in a button's <code>onClick</code>
<code>useState<Todo null>(null)</code>	State that starts empty and holds a <code>Todo</code> later
<code>{ params: Promise<{ id: string }> }</code>	Props of a Next.js page or route in an <code>[id]</code> folder
<code>Request</code> / <code>NextRequest</code> , <code>NextResponse</code>	Route handler input and output (<code>app/api/.../route.ts</code>)

TIP

Don't memorise event types. Write the handler inline once, e.g. `onChange={ (e) => ... }`, hover over `e` in VS Code, and copy the type it shows you.

15 The tsconfig settings that matter

Setting	Effect
<code>"strict": true</code>	Turns on the important checks, including the two below. Keep it on.
<code>strictNullChecks</code> (part of <code>strict</code>)	<code>null</code> and <code>undefined</code> aren't allowed unless the type says so. Catches the #1 JavaScript bug: "cannot read properties of undefined".
<code>noImplicitAny</code> (part of <code>strict</code>)	A parameter with no type and nothing to infer from is an error, not a silent <code>any</code> .
<code>"paths": { "@/*": ["../.*"] }</code>	The <code>@/</code> import shortcut.

Part 6: Errors, exercises, interviews

Reading error messages

These are real messages from compiling broken code. Each has a code (TS + number) you can search for.

Code that's wrong	What tsc says, and the fix
<pre>let city = "Manila"; city = 42;</pre>	TS2322: Type 'number' is not assignable to type 'string'. The variable was inferred as a string. Use a string, or type it <code>string number</code> if both are really allowed.
<pre>const a: Todo = { id: 1, text: "Walk dog" };</pre>	TS2741: Property 'done' is missing in type '{ id: number; text: string; }' but required in type 'Todo'. Add <code>done</code>, or make it optional in the type (<code>done?: boolean</code>).

<pre>{ id: 2, text: "Feed cat", done: false, colour: "red" }</pre>	TS2353: Object literal may only specify known properties, and 'colour' does not exist in type 'Todo'. A typo or a field the type doesn't have.
<pre>return todo.dueDate.length;</pre>	TS18048: 'todo.dueDate' is possibly 'undefined'. It's optional. Check first (<code>if (todo.dueDate)</code>) or use <code>todo.dueDate?.length ?? 0</code> .
<pre>function echo(x) { return x; }</pre>	TS7006: Parameter 'x' implicitly has an 'any' type. Give the parameter a type: <code>(x: string)</code> .
<pre>todos[0].txt</pre>	TS2551: Property 'txt' does not exist on type 'Todo'. Did you mean 'text'? A typo. TypeScript even suggests the fix.
<pre>function getText(todo: Todo): string { if (todo.done) return todo.text; }</pre>	TS2366: Function lacks ending return statement and return type does not include 'undefined'. Some paths return nothing. Add a final <code>return</code> , or change the return type to <code>string undefined</code> .

TIP

Read the message **right to left**: "X is not assignable to Y" = "you promised Y, but gave X".

Exercises

Write these in `guide/practice/typescript-basics.mts`, then run `node` and `npx tsc --noEmit`.

1. Write `type Priority = 1 | 2 | 3` and add `priority: Priority` to `Todo`. What happens if you set 5?
2. Write `countOpen(todos: Todo[]): number` that returns how many are not done.
3. Write `type TodoPatch = Partial<Omit<Todo, "id" | "createdBy">>`. Which fields can it hold?
4. Write a generic `last<T>(items: T[]): T | undefined`.
5. Write `type ApiResult<T> = { ok: true; data: T } | { ok: false; error: string }` and a function that turns it into a message.

Answers

TypeScript

```
// 1. Setting priority: 5 → TS2322: Type '5' is not assignable to type 'Priority'.
type Priority = 1 | 2 | 3;

// 2.
function countOpen(todos: Todo[]): number {
  return todos.filter((t) => !t.done).length;
}

// 3. text, done, dueDate (and priority, after exercise 1), all optional; id and createdBy removed

// 4.
function last<T>(items: T[]): T | undefined {
  return items[items.length - 1];
}

// 5.
type ApiResult<T> = { ok: true; data: T } | { ok: false; error: string };
function message<T>(result: ApiResult<T>): string {
  return result.ok ? "Success" : `Error: ${result.error}`;
}
```

Interview questions, short answers

Question	Short answer
Why use TypeScript?	Catches mistakes before running the code, gives autocomplete, and documents the shape of data. Refactoring is safer because the checker finds every place that breaks.
Does TypeScript check at runtime?	No. Types are erased. Data from outside (APIs, forms) still needs runtime checks or a validation library like Zod.
<code>type</code> vs <code>interface</code> ?	Both describe objects. Only <code>type</code> can do unions; <code>interface</code> uses <code>extends</code> and can be reopened. Pick one style and be consistent.
<code>any</code> vs <code>unknown</code> ?	<code>any</code> disables checking. <code>unknown</code> forces you to narrow before use. Prefer <code>unknown</code> .
What are generics?	Type parameters, like <code>Array<T></code> or <code>Promise<T></code> , so one function or type works with many types and stays safe.
What is narrowing?	TypeScript refining a type from your checks (<code>typeof</code> , <code>in</code> , <code>===</code> , truthiness), e.g. <code>string number</code> becomes <code>number</code> inside <code>if (typeof x === "number")</code> .
What does <code>?</code> do in a type?	Marks a property optional: it may be missing, so its type includes <code>undefined</code> .

INTERVIEW ANGLE

A good line to have ready: *"I keep types at the boundaries: the database row type in the model, request checks in the API routes, and props on components. Inside functions I let TypeScript infer."*